

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM HUẾ

BÁO CÁO DỰ ÁN

Môn: Tích hợp CNTT trong dạy học toán

SỬ DỤNG SYMPY
ĐỂ GIẢI QUYẾT
CÁC BÀI TOÁN VỀ MA TRẬN

Giảng viên hướng dẫn: Thầy Nguyễn Đăng Minh Phúc

Lớp: Toán 4A

Nhóm thực hiện: Nhóm 8

Thành viên nhóm:

- Đinh Thị Kim Anh
- Phạm Thị Bích Ngân
- Trần Yến Vi
- Yn Lân

Chương 1

Mở đầu

1.1 Lý do chọn đề tài

Đại số tuyến tính nói chung và lý thuyết ma trận nói riêng là một trong những nền tảng toán học vô cùng quan trọng, được ứng dụng rộng rãi trong nhiều lĩnh vực khoa học kỹ thuật như trí tuệ nhân tạo, đồ họa máy tính, mật mã học, vật lý học và kinh tế học. Tuy nhiên, việc tính toán ma trận bằng phương pháp thủ công (như nhân các ma trận kích thước lớn, tìm ma trận nghịch đảo, hay biến đổi sơ cấp đưa về dạng bậc thang) thường đòi hỏi nhiều thời gian, công sức và rất dễ dẫn đến sai sót.

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ, việc ứng dụng các ngôn ngữ lập trình vào quá trình học tập và nghiên cứu Toán học đã trở thành một xu hướng tất yếu. Ngôn ngữ lập trình Python, với cú pháp thân thiện và hệ sinh thái thư viện phong phú, đang đóng vai trò là một công cụ hỗ trợ đắc lực. Thay vì chỉ tiếp cận Toán học qua các định lý và phép tính trên giấy, việc kết hợp lập trình giúp người học và người nghiên cứu có thể kiểm chứng kết quả nhanh chóng, tự động hóa các bước tính toán lặp đi lặp lại và hiểu sâu hơn về bản chất thuật toán.

Xuất phát từ nhu cầu thực tiễn đó, chúng tôi quyết định thực hiện dự án với đề tài "**Sử dụng SymPy để giải quyết các bài toán về ma trận**". Đề tài này không chỉ giúp củng cố kiến thức nền tảng về đại số tuyến tính mà còn mang đến một góc nhìn hiện đại, hướng dẫn cách sử dụng công cụ tính toán ký hiệu (symbolic computation) để giải quyết các bài toán Toán học một cách chính xác và hiệu quả nhất.

1.2 Giới thiệu về dự án

1.2.1 Mục tiêu của dự án

Dự án được xây dựng với các mục tiêu cốt lõi sau:

- **Hệ thống hóa kiến thức:** Ôn tập và trình bày lại một cách logic các nền tảng lý thuyết cơ bản về ma trận, hệ phương trình tuyến tính và các phép biến đổi liên quan.
- **Ứng dụng công nghệ:** Hướng dẫn chi tiết cách sử dụng các hàm và phương thức có sẵn trong thư viện SymPy của Python để mô phỏng và giải quyết các bài toán ma trận tương ứng.

- **Đối chiếu và xác minh:** So sánh quá trình tính toán thủ công với kết quả chạy bằng mã máy tính, qua đó làm nổi bật tính chính xác và ưu việt của việc ứng dụng lập trình vào Toán học.

1.2.2 Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Các bài toán cơ bản về ma trận (cộng, trừ, nhân, chuyển vị), định thức, ma trận nghịch đảo, phép biến đổi sơ cấp trên hàng, hạng của ma trận và bài toán giải hệ phương trình tuyến tính.
- **Phạm vi ứng dụng:** Dự án hướng tới những sinh viên, học sinh đang học môn Đại số tuyến tính, cũng như các giảng viên, giáo viên cần một công cụ để kiểm tra chéo đáp án hoặc tự động hóa việc tạo đề thi, bài tập.

1.3 Giới thiệu về thư viện SymPy

SymPy (Symbolic Python) là một thư viện nguồn mở của ngôn ngữ lập trình Python, được thiết kế chuyên biệt để thực hiện các phép tính toán học ký hiệu (symbolic mathematics). Khác với các thư viện tính toán số học (numerical computation) thông thường, SymPy giữ nguyên bản chất đại số của các biểu thức, cho phép tính toán với các hằng số chính xác, phân số và các biến ký hiệu mà không bị làm tròn thành số thập phân. Đặc biệt, thư viện này còn hỗ trợ xuất kết quả dưới dạng mã $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$, giúp việc trình bày công thức vô cùng tiện lợi.

1.3.1 Cài đặt và sử dụng SymPy

Để bắt đầu sử dụng thư viện SymPy, môi trường máy tính cần được cài đặt sẵn ngôn ngữ Python. Sau đó, thư viện có thể được cài đặt dễ dàng thông qua công cụ quản lý gói `pip` bằng lệnh sau trên Terminal (hoặc Command Prompt):

```
pip install sympy
```

Khi thư viện đã được cài đặt, ta chỉ cần gọi nó vào chương trình Python để bắt đầu sử dụng:

```
import sympy as sp
```

Kể từ đây, mọi sức mạnh tính toán đại số của SymPy đã sẵn sàng để được áp dụng vào việc giải quyết các bài toán ma trận được trình bày trong các chương tiếp theo của báo cáo.

Chương 2

Giới thiệu các lệnh SymPy được sử dụng

Trong chương này, chúng tôi tổng hợp và giới thiệu cú pháp cũng như chức năng của các lệnh trong thư viện SymPy đã được áp dụng để giải quyết các bài toán ma trận trong dự án. Việc nắm vững các lệnh này là cơ sở để hiểu rõ phần cài đặt thuật toán và mã nguồn ở các chương sau.

2.1 Khai báo thư viện và khởi tạo ma trận

- **Khai báo thư viện:** Nhập thư viện SymPy vào môi trường làm việc và gán tên viết tắt là `sp` để tiện lợi khi gọi các hàm.

```
import sympy as sp
```

- **Tạo ma trận thông thường:** Sử dụng đối tượng `sp.Matrix()` với đầu vào là một danh sách (list) chứa các hàng của ma trận.

```
A = sp.Matrix([
    [1, 2],
    [3, 4]
])
```

- **Tạo ma trận đặc biệt:** `sp.zeros(m, n)` để tạo ma trận không kích thước $m \times n$, và `sp.eye(n)` để tạo ma trận đơn vị cấp n .

```
O = sp.zeros(2, 2)
I = sp.eye(3)
```

- **Các hàm hỗ trợ toán học:** Khai báo số hữu tỉ (phân số) để hệ thống giữ nguyên độ chính xác tuyệt đối mà không bị làm tròn, đồng thời khai báo biến ký hiệu toán học đại số.

```
# Khai bao phan so 1/3
frac = sp.Rational(1, 3)
```

```
# Khai bao bien x, y, z
x, y, z = sp.symbols('x y z')
```

2.2 Các phép toán ma trận cơ bản

Các phép toán đại số trên ma trận được SymPy hỗ trợ rất trực quan thông qua các toán tử cơ bản và phương thức tích hợp sẵn của đối tượng `Matrix`.

- **Cộng, trừ, nhân:** Sử dụng trực tiếp các ký hiệu toán tử lần lượt là `+`, `-`, `*`.

```
C = A + B
D = A - B
E = 4 * A      # Nhân ma tran voi so vo huong
F = A * B      # Nhân hai ma tran
```

- **Các đặc trưng của ma trận:** SymPy cung cấp các phương thức để tìm chuyển vị (`.T`), định thức (`.det()`), và ma trận nghịch đảo (`.inv()`).

```
A_T = A.T      # Ma tran chuyen vi
d = A.det()     # Dinh thuc
A_inv = A.inv() # Ma tran Nghich dao
```

2.3 Biến đổi sơ cấp và hạng của ma trận

- **Tạo bản sao ma trận:** Các lệnh biến đổi sơ cấp thường làm thay đổi trực tiếp lên đối tượng. Do đó, dùng phương thức `.copy()` để tạo bản sao nếu muốn giữ lại ma trận gốc.

```
B = A.copy()
```

- **Biến đổi sơ cấp trên hàng:** Dùng phương thức `row_swap` để đổi chỗ và `row_op` kết hợp với hàm `lambda` để thực hiện tính toán trên hàng. (Lưu ý: chỉ số hàng/cột bắt đầu từ 0).

```
# Doi cho hang 1 va hang 3
A.row_swap(0, 2)

# Nhan hang 2 voi 2
A.row_op(1, lambda val, j: 2 * val)

# Cong (-2) lan hang 1 vao hang 2
A.row_op(1, lambda val, j: val - 2 * A[0, j])
```

- **Dạng bậc thang và hạng:** Chuyển ma trận về dạng bậc thang (`echelon_form()`), bậc thang rút gọn RREF (`rref()`) và tính hạng của ma trận (`rank()`). Lệnh `rref()` trả về ma trận kết quả và vị trí các cột trụ.

```
E = A.echelon_form()
R, pivot_cols = A.rref()
r = A.rank()
```

2.4 Giải hệ phương trình tuyến tính

SymPy cung cấp nhiều công cụ linh hoạt để xử lý bài toán hệ phương trình tuyến tính dưới dạng ma trận $AX = B$.

- **Tạo ma trận mở rộng:** Dùng phương thức `row_join()` để ghép ma trận cột hệ số tự do B vào bên phải ma trận hệ số A .

```
Ab = A.row_join(B)
```

- **Phương pháp Gauss-Jordan:** Giải và trả về ngay ma trận nghiệm kèm theo các tham số (nếu có).

```
solution, params = A.gauss_jordan_solve(B)
```

- **Giải hệ tổng quát `linsolve()`:** Đây là hàm mạnh mẽ nhất để giải hệ tuyến tính, trả về dạng tập nghiệm. Hàm này hoạt động tốt cho cả trường hợp hệ vô nghiệm (trả về tập rỗng), nghiệm duy nhất hoặc vô số nghiệm.

```
sol_set = sp.linsolve((A, B))
```

- **Kiểm tra nghiệm:** Dùng phương thức `.subs()` để thay thế (substitution) tập giá trị nghiệm tìm được ngược trở lại vào phương trình ban đầu nhằm kiểm tra tính đúng đắn.

```
# Thay the tap gia tri vao bieu thuc eq
result = eq.subs({x: sp.Rational(1, 3), y: sp.Rational(10, 9)
})
```

Chương 3

Giải một số bài toán về ma trận bằng SymPy

Trong chương này, chúng tôi sử dụng thư viện **SymPy** trong ngôn ngữ lập trình Python để giải quyết một số bài toán cơ bản về ma trận. Các nội dung được tập trung nghiên cứu gồm: thực hiện các phép biến đổi sơ cấp trên ma trận, xác định hạng của ma trận và giải hệ phương trình tuyến tính.

Việc sử dụng **SymPy** giúp thực hiện các phép tính ma trận một cách nhanh chóng, chính xác và hạn chế những sai sót có thể xảy ra khi tính toán bằng tay. Bên cạnh việc sử dụng các hàm có sẵn của thư viện, các ví dụ trong chương còn đối chiếu với phương pháp tính toán truyền thống nhằm làm rõ bản chất toán học của từng bài toán.

Trước khi thực hiện các phép tính, cần khai báo thư viện **SymPy** bằng lệnh:

```
import sympy as sp
```

Trong toàn bộ chương này, ký hiệu **sp** được sử dụng thay cho tên đầy đủ của thư viện **SymPy**.

3.1 Khai báo ma trận trong SymPy

Để tạo một ma trận trong **SymPy**, sử dụng đối tượng **Matrix**.

Ví dụ, xét ma trận

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}.$$

Ta khai báo trong Python như sau:

```
import sympy as sp

A = sp.Matrix([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

print(A)
```

Kết quả:

```
Matrix([
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]])
```

Có thể truy cập phần tử ở hàng i , cột j của ma trận bằng cú pháp:

```
A[i, j]
```

Trong Python, chỉ số được bắt đầu từ 0. Do đó, phần tử a_{11} được gọi bằng:

```
A[0, 0]
```

3.2 Phép toán cơ bản trên ma trận

3.2.1 Cộng trừ ma trận

a. Nhắc lại lý thuyết

Cho hai ma trận $A_{mn} = [a_{ij}]_{mn}$ và $B_{mn} = [b_{ij}]_{mn}$ có cùng kích thước $m \times n$. Khi đó, ma trận tổng (hoặc hiệu) $C_{mn} = A \pm B = [c_{ij}]_{mn}$ là một ma trận mới có cùng kích thước với A và B . Trong đó:

$$c_{ij} = a_{ij} \pm b_{ij}, \forall i = \overline{1, m}; j = \overline{1, n}.$$

- $a_{ij} \rightarrow$ Phần tử nằm ở vị trí hàng i , cột j của ma trận A .
- $b_{ij} \rightarrow$ Phần tử nằm ở vị trí hàng i , cột j của ma trận B .

b. Ví dụ

Tính:

$$\begin{bmatrix} 1 & 4 & -2 \\ 3 & 0 & 5 \end{bmatrix} + \begin{bmatrix} -1 & 2 & 3 \\ 4 & -3 & 1 \end{bmatrix}$$

Trong Sympy:

```
import sympy as sp

A = sp.Matrix([
    [1, 4, -2],
    [3, 0, 5]
])

B = sp.Matrix([
    [-1, 2, 3],
    [4, -3, 1]
])
```

```
print(A + B)
```

Kết quả:

```
Matrix([
[0,  6, 1],
[7, -3, 6]])
```

Tính:

$$\begin{bmatrix} 5 & -1 \\ 2 & 7 \\ 0 & 4 \end{bmatrix} - \begin{bmatrix} 2 & 3 \\ -1 & 4 \\ 1 & 0 \end{bmatrix}$$

Trong Sympy:

```
A = sp.Matrix([
    [5, -1],
    [2, 7],
    [0, 4]
])

B = sp.Matrix([
    [2, 3],
    [-1, 4],
    [1, 0]
])

print(A - B)
```

Kết quả:

```
Matrix([
[ 3, -4],
[ 3,  3],
[-1,  4]])
```

3.2.2 Nhân ma trận với một vô hướng

a. Nhắc lại lý thuyết

Cho ma trận $A_{mn} = [a_{ij}]_{mn}$ và một số vô hướng k (thường là một số thực hoặc số phức). Khi đó tích của số vô hướng k với ma trận A , ký hiệu là kA , là một ma trận mới $B_{mn} = [c_{ij}]_{mn}$ có cùng kích thước với A . Trong đó:

$$c_{ij} = k \cdot a_{ij}, \forall i = \overline{1, m}; j = \overline{1, n}.$$

- $k \rightarrow$ Số vô hướng đem nhân.
- $a_{ij} \rightarrow$ Phần tử nằm ở hàng i và cột j của ma trận gốc A .

b. Ví dụ

Tính:

$$4 \begin{bmatrix} 2 & -1 & 0 \\ 3 & 5 & -2 \end{bmatrix}$$

Trong Sympy:

```
A = sp.Matrix([
    [2, -1, 0],
    [3, 5, -2]
])

print(4 * A)
```

Kết quả:

```
Matrix([
[ 8, -4,  0],
[12, 20, -8]])
```

Tính:

$$-\frac{1}{3} \begin{bmatrix} -6 & 9 \\ 3 & 0 \\ -12 & 15 \end{bmatrix}$$

Trong Sympy:

```
A = sp.Matrix([
    [-6, 9],
    [3, 0],
    [-12, 15]
])

# Dùng sp.Rational để giữ nguyên phân số thay vì số thập phân
print(sp.Rational(-1, 3) * A)
```

Kết quả:

```
Matrix([
[ 2, -3],
```

```
[-1, 0],  
[ 4, -5]]])
```

3.2.3 Nhân hai ma trận

a. Nhắc lại lý thuyết

Cho hai ma trận A_{mp} và B_{pn} . Khi đó ma trận $C_{mn} = A_{mp}B_{pn} = [c_{ij}]_{mn}$ gọi là tích của hai ma trận A, B . Trong đó:

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{ip}b_{pj}, \forall i = \overline{1, m}; j = \overline{1, n}.$$

- $a_{i1}, a_{i2}, \dots, a_{ip} \rightarrow$ Hàng thứ i của ma trận A .
- $b_{1j}, b_{2j}, \dots, b_{pj} \rightarrow$ Cột thứ j của ma trận B .

Như vậy c_{ij} = hàng thứ i của ma trận A nhân tương ứng với cột thứ j của ma trận B rồi cộng lại.

b. Ví dụ

Tính tích của hai ma trận vuông cấp 2:

$$\begin{bmatrix} 2 & -1 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 5 \\ -2 & 0 \end{bmatrix}$$

Trong SymPy:

```
A = sp.Matrix([  
    [2, -1],  
    [3, 4]  
)  
  
B = sp.Matrix([  
    [1, 5],  
    [-2, 0]  
)  
  
print(A * B)
```

Kết quả:

```
Matrix([  
[ 4, 10],  
[-5, 15]])
```

Tính tích của ma trận cỡ 2×3 và ma trận cỡ 3×2 :

$$\begin{bmatrix} 1 & -2 & 3 \\ 0 & 4 & -1 \end{bmatrix} \begin{bmatrix} 2 & -1 \\ 1 & 3 \\ -2 & 0 \end{bmatrix}$$

Trong Sympy:

```
A = sp.Matrix([
    [1, -2, 3],
    [0, 4, -1]
])

B = sp.Matrix([
    [2, -1],
    [1, 3],
    [-2, 0]
])

print(A * B)
```

Kết quả:

```
Matrix([
[-6, -7],
[ 6, 12]])
```

3.2.4 Phép chuyển vị (A^T)

a. Nhắc lại lý thuyết

Cho ma trận $A_{mn} = [a_{ij}]_{mn}$. Khi đó ma trận $A_{nm}^T = [c_{ij}]_{nm}$ gọi là ma trận chuyển vị của ma trận A . Trong đó:

$$c_{ij} = a_{ji}, \forall i = \overline{1, n}; j = \overline{1, m}.$$

- $a_{j1}, a_{j2}, \dots, a_{jn} \rightarrow$ Hàng thứ j của ma trận A sẽ trở thành cột thứ j của ma trận A^T .
- $a_{1i}, a_{2i}, \dots, a_{mi} \rightarrow$ Cột thứ i của ma trận A sẽ trở thành hàng thứ i của ma trận A^T .

Như vậy c_{ij} = phần tử nằm ở vị trí giao giữa hàng j và cột i của ma trận gốc A . Quá trình này thực chất là đổi lần lượt các hàng của ma trận A thành các cột tương ứng để tạo ra ma trận A^T .

b. Ví dụ

Tìm ma trận chuyển vị A^T của ma trận A sau:

$$A = \begin{bmatrix} 3 & -1 & 5 \\ 0 & 2 & -4 \end{bmatrix}$$

Trong Sympy:

```
A = sp.Matrix([
    [3, -1, 5],
    [0, 2, -4]
])

print(A.T)
```

Kết quả:

```
Matrix([
[ 3,  0],
[-1,  2],
[ 5, -4]])
```

Tìm ma trận chuyển vị B^T của ma trận B sau:

$$B = \begin{bmatrix} 1 & 0 \\ -3 & 7 \\ 2 & 5 \\ 4 & -1 \end{bmatrix}$$

Trong Sympy:

```
B = sp.Matrix([
    [1, 0],
    [-3, 7],
    [2, 5],
    [4, -1]
])

print(B.T)
```

Kết quả:

```
Matrix([
[1, -3, 2,  4],
[0,  7, 5, -1]])
```

3.3 Định thức và ma trận nghịch đảo

3.3.1 Định thức

a. Nhắc lại lý thuyết

Cho ma trận vuông $A_n = [a_{ij}]_n$. Khi đó định thức của ma trận A , ký hiệu là $\det(A)$ hoặc $|A|$, là một số vô hướng gắn liền với ma trận đó. Trong đó, công thức tính định thức bằng cách khai triển theo hàng thứ i bất kỳ ($i = \overline{1, n}$) là:

$$\det(A) = a_{i1}C_{i1} + a_{i2}C_{i2} + \cdots + a_{in}C_{in} = \sum_{j=1}^n a_{ij}C_{ij}.$$

- $a_{i1}, a_{i2}, \dots, a_{in} \rightarrow$ Các phần tử nằm trên hàng thứ i của ma trận A .
- $C_{ij} \rightarrow$ Phần bù đại số của phần tử a_{ij} , được tính bằng công thức $(-1)^{i+j}M_{ij}$.
- $M_{ij} \rightarrow$ Định thức con cấp $n - 1$, thu được bằng cách xóa đi hàng i và cột j của ma trận gốc A .

b. Ví dụ

Tính định thức của ma trận A cấp 3×3 sau:

$$A = \begin{bmatrix} 2 & -1 & 3 \\ 4 & 1 & 5 \\ -2 & 0 & 1 \end{bmatrix}$$

Trong Sympy:

```
A = sp.Matrix([
    [2, -1, 3],
    [4, 1, 5],
    [-2, 0, 1]
])

print(A.det())
```

Kết quả:

22

Tính định thức của ma trận A cấp 4×4 sau:

$$A = \begin{bmatrix} 2 & 1 & 0 & -1 \\ -1 & 3 & 2 & 0 \\ 0 & 4 & -2 & 1 \\ 3 & 0 & 1 & -3 \end{bmatrix}$$

Trong Sympy:

```
A = sp.Matrix([
    [2, 1, 0, -1],
    [-1, 3, 2, 0],
    [0, 4, -2, 1],
    [3, 0, 1, -3]
])
```

```
] )
```

```
print(A.det())
```

Kết quả:

```
123
```

3.3.2 Ma trận nghịch đảo

a. Nhắc lại lý thuyết

Cho ma trận vuông $A_n = [a_{ij}]_n$ thỏa mãn điều kiện định thức $\det(A) \neq 0$. Khi đó ma trận $A_n^{-1} = [c_{ij}]_n$ thỏa mãn $A \cdot A^{-1} = A^{-1} \cdot A = I_n$ gọi là ma trận nghịch đảo của ma trận A . Trong đó, công thức tính phần tử là:

$$c_{ij} = \frac{1}{\det(A)} C_{ji}, \forall i = \overline{1, n}; j = \overline{1, n}.$$

- $\det(A) \rightarrow$ Định thức của ma trận gốc A .
- $C_{ji} \rightarrow$ Phần bù đại số của phần tử a_{ji} trong ma trận A (được tính bằng $(-1)^{j+i} M_{ji}$, với M_{ji} là định thức con khi bỏ đi hàng j và cột i).
- $I_n \rightarrow$ Ma trận đơn vị cấp n (các phần tử trên đường chéo chính bằng 1, còn lại bằng 0).

b. Ví dụ

Tìm ma trận nghịch đảo của ma trận A cấp 2×2 sau:

$$A = \begin{bmatrix} 2 & 5 \\ 1 & 3 \end{bmatrix}$$

Trong Sympy:

```
A = sp.Matrix([
    [2, 5],
    [1, 3]
])
```

```
print(A.inv())
```

Kết quả:

```
Matrix([
[ 3, -5],
[-1,  2]])
```

Tìm ma trận nghịch đảo của ma trận B cấp 3×3 sau:

$$B = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & -1 \end{bmatrix}$$

Trong Sympy:

```
B = sp.Matrix([
    [1, 2, 0],
    [0, 1, 1],
    [1, 0, -1]
])

print(B.inv())
```

Kết quả:

```
Matrix([
  [-1,  2,  2],
  [ 1, -1, -1],
  [-1,  2,  1]])
```

3.4 Các phép biến đổi sơ cấp trên ma trận

3.4.1 Nhắc lại lý thuyết

Các phép biến đổi sơ cấp trên hàng của ma trận gồm ba loại cơ bản:

1. Đổi chỗ hai hàng của ma trận.
2. Nhân một hàng với một số khác không.
3. Cộng vào một hàng một bội của một hàng khác.

Cụ thể, với ma trận

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix},$$

ta có các phép biến đổi sơ cấp sau.

Đổi chỗ hai hàng

Đổi chỗ hai hàng R_i và R_j được ký hiệu bởi

$$R_i \leftrightarrow R_j.$$

Ví dụ:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \longrightarrow \begin{pmatrix} 3 & 4 \\ 1 & 2 \end{pmatrix}.$$

Nhân một hàng với một số khác không

Với $\lambda \neq 0$, phép biến đổi

$$R_i \longrightarrow \lambda R_i$$

là một phép biến đổi sơ cấp.

Ví dụ:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \longrightarrow \begin{pmatrix} 1 & 2 \\ 6 & 8 \end{pmatrix}$$

khi thực hiện $R_2 \rightarrow 2R_2$.

Cộng một bội của hàng này vào hàng khác

Phép biến đổi

$$R_i \longrightarrow R_i + \lambda R_j, \quad i \neq j$$

cũng là một phép biến đổi sơ cấp.

Ví dụ:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \longrightarrow \begin{pmatrix} 1 & 2 \\ 5 & 8 \end{pmatrix}$$

khi thực hiện

$$R_2 \longrightarrow R_2 + 2R_1.$$

Các phép biến đổi sơ cấp có vai trò quan trọng trong việc đưa ma trận về dạng bậc thang hoặc dạng bậc thang rút gọn. Đồng thời, các phép biến đổi sơ cấp trên hàng không làm thay đổi hạng của ma trận.

3.4.2 Đổi chỗ hai hàng

Trong SymPy, phương thức

```
row_swap(i, j)
```

được sử dụng để đổi chỗ hàng i và hàng j .

Xét ma trận

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}.$$

Thực hiện đổi chỗ hàng thứ nhất và hàng thứ ba:

```
import sympy as sp

A = sp.Matrix([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
```

```

])

A.row_swap(0, 2)

print(A)

```

Kết quả thu được:

```

Matrix([
[7, 8, 9],
[4, 5, 6],
[1, 2, 3]])

```

Như vậy, lệnh

```
A.row_swap(0, 2)
```

tương ứng với phép biến đổi toán học

$$R_1 \leftrightarrow R_3.$$

3.4.3 Nhân một hàng với một số khác không

Để nhân một hàng với một số, có thể sử dụng phương thức `row_op()`.

Xét ma trận

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}.$$

Thực hiện phép biến đổi

$$R_2 \longrightarrow 2R_2.$$

Trong Python:

```

import sympy as sp

A = sp.Matrix([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

A.row_op(1, lambda value, j: 2*value)

print(A)

```

Kết quả:

```

Matrix([
[1, 2, 3],
[8, 10, 12],
[7, 8, 9]])

```

Ở đây, số 1 trong `row_op(1, ...)` biểu thị hàng thứ hai vì Python đánh số từ 0.

3.4.4 Cộng một bội của hàng vào hàng khác

Xét phép biến đổi

$$R_2 \longrightarrow R_2 - 2R_1.$$

Ta có thể thực hiện bằng:

```
import sympy as sp

A = sp.Matrix([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

A.row_op(1, lambda value, j: value - 2*A[0, j])

print(A)
```

Kết quả là

```
Matrix([
[1, 2, 3],
[2, 1, 0],
[7, 8, 9]])
```

Do đó, `row_op()` cho phép mô phỏng trực tiếp các phép biến đổi sơ cấp thường được sử dụng trong phương pháp Gauss.

3.4.5 Đưa ma trận về dạng bậc thang

Một trong những ứng dụng quan trọng của phép biến đổi sơ cấp là đưa ma trận về dạng bậc thang.

Trong SymPy, có thể sử dụng phương thức:

```
echelon_form()
```

Ví dụ:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 2 \end{pmatrix}.$$

Ta thực hiện:

```
import sympy as sp

A = sp.Matrix([
    [1, 2, 3],
    [2, 4, 6],
    [1, 1, 2]
```

```

])

E = A.echelon_form()

print(E)

```

Kết quả có dạng

$$\begin{pmatrix} 1 & 2 & 3 \\ 0 & -1 & -1 \\ 0 & 0 & 0 \end{pmatrix}.$$

Ma trận trên là ma trận dạng bậc thang.

3.4.6 Đưa ma trận về dạng bậc thang rút gọn

Ngoài dạng bậc thang, SymPy còn cho phép đưa ma trận về dạng bậc thang rút gọn bằng phương thức `rref()`.

Cú pháp:

```
R, pivot_columns = A.rref()
```

Trong đó:

- `R` là ma trận dạng bậc thang rút gọn;
- `pivot_columns` là các cột chứa phần tử trụ.

Ví dụ:

```

import sympy as sp

A = sp.Matrix([
    [1, 2, 3],
    [2, 4, 6],
    [1, 1, 2]
])

R, pivot_columns = A.rref()

print(R)
print(pivot_columns)

```

Kết quả:

$$R = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{pmatrix}.$$

Các cột trụ được trả về dưới dạng:

```
(0, 1)
```

Do đó, ma trận có hai cột trụ.

3.4.7 Nhận xét về các phép biến đổi sơ cấp

Các phép biến đổi sơ cấp là công cụ quan trọng trong việc biến đổi ma trận về những dạng thuận tiện cho việc tính toán. Trong đó, việc đưa ma trận về dạng bậc thang hoặc dạng bậc thang rút gọn giúp xác định hạng ma trận và giải hệ phương trình tuyến tính.

Khi sử dụng **SymPy**, người sử dụng không nhất thiết phải thực hiện thủ công từng phép biến đổi. Tuy nhiên, việc hiểu các phép biến đổi này vẫn cần thiết để giải thích ý nghĩa toán học của kết quả do chương trình đưa ra.

3.5 Hạng của ma trận

3.5.1 Khái niệm hạng của ma trận

Hạng của ma trận là số chiều của không gian hàng của ma trận, tương đương với số chiều của không gian cột.

Ký hiệu hạng của ma trận A là

$$\text{rank}(A).$$

Nếu đưa ma trận A về dạng bậc thang bằng các phép biến đổi sơ cấp trên hàng thì hạng của A bằng số hàng khác không trong ma trận dạng bậc thang.

Ngoài ra, hạng của ma trận cũng bằng số phần tử trụ khi đưa ma trận về dạng bậc thang rút gọn.

3.5.2 Tính hạng ma trận bằng SymPy

Trong **SymPy**, phương thức

```
rank()
```

được sử dụng để xác định hạng của ma trận.

Ví dụ:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 2 \end{pmatrix}.$$

Ta thực hiện:

```
import sympy as sp

A = sp.Matrix([
    [1, 2, 3],
    [2, 4, 6],
    [1, 1, 2]
])

r = A.rank()

print(r)
```

Kết quả:

2

Vậy

$$\text{rank}(A) = 2.$$

3.5.3 Xác định hạng thông qua dạng bậc thang rút gọn

Ta cũng có thể xác định hạng của ma trận thông qua các cột trụ.

```
R, pivot_columns = A.rref()

print("Ma tran RREF:")
print(R)

print("Cac cot tru:")
print(pivot_columns)

print("Hang cua ma tran:")
print(len(pivot_columns))
```

Do số cột trụ bằng 2 nên

$$\text{rank}(A) = 2.$$

Cách làm này cho thấy mối liên hệ giữa dạng bậc thang rút gọn và hạng của ma trận.

3.5.4 Một số trường hợp đặc biệt

Ma trận không

Với ma trận không

$$O = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix},$$

ta có

$$\text{rank}(O) = 0.$$

Trong SymPy:

```
O = sp.zeros(2, 2)

print(O.rank())
```

Kết quả:

0

Ma trận đơn vị

Xét ma trận đơn vị cấp 3:

$$I_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Ta có

$$\text{rank}(I_3) = 3.$$

Trong SymPy:

```
I = sp.eye(3)
print(I.rank())
```

Kết quả:

3

3.5.5 Ý nghĩa của hạng ma trận

Hạng của ma trận cho biết số lượng hàng hoặc cột độc lập tuyến tính tối đa của ma trận.

Chẳng hạn, với

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{pmatrix},$$

ta nhận thấy

$$R_2 = 2R_1, \quad R_3 = 3R_1.$$

Do đó chỉ có một hàng độc lập tuyến tính, suy ra

$$\text{rank}(A) = 1.$$

Trong SymPy:

```
A = sp.Matrix([
    [1, 2, 3],
    [2, 4, 6],
    [3, 6, 9]
])
print(A.rank())
```

Kết quả:

1

Như vậy, việc tính hạng bằng SymPy cho phép kiểm tra nhanh kết quả thu được từ quá trình biến đổi sơ cấp.

3.6 Giải hệ phương trình tuyến tính bằng SymPy

3.6.1 Cơ sở lý thuyết

Xét hệ phương trình tuyến tính n ẩn:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2, \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m. \end{cases}$$

Hệ phương trình có thể được viết dưới dạng ma trận:

$$AX = B,$$

trong đó

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix},$$
$$X = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \quad B = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}.$$

Ma trận mở rộng của hệ là

$$\overline{A} = (A \mid B).$$

3.6.2 Định lý Rouché–Capelli

Định lý 3.1 (Rouché–Capelli). *Hệ phương trình tuyến tính*

$$AX = B$$

có nghiệm khi và chỉ khi

$$\text{rank}(A) = \text{rank}(\overline{A}).$$

Nếu hệ có nghiệm, ta có ba trường hợp:

- Nếu

$$\text{rank}(A) = \text{rank}(\overline{A}) = n,$$

thì hệ có nghiệm duy nhất.

- Nếu

$$\text{rank}(A) = \text{rank}(\overline{A}) < n,$$

thì hệ có vô số nghiệm.

- Nếu

$$\text{rank}(A) < \text{rank}(\overline{A}),$$

thì hệ vô nghiệm.

Định lý này có thể được kiểm tra trực tiếp bằng SymPy thông qua phương thức `rank()`.

3.6.3 Khai báo hệ phương trình trong SymPy

Xét hệ phương trình:

$$\begin{cases} x + 2y + z = 4, \\ 2x - y + z = 1, \\ x + y - z = 0. \end{cases}$$

Ma trận hệ số là

$$A = \begin{pmatrix} 1 & 2 & 1 \\ 2 & -1 & 1 \\ 1 & 1 & -1 \end{pmatrix}$$

và ma trận cột hệ số tự do là

$$B = \begin{pmatrix} 4 \\ 1 \\ 0 \end{pmatrix}.$$

Trong SymPy:

```
import sympy as sp

A = sp.Matrix([
    [1, 2, 1],
    [2, -1, 1],
```

```

    [1, 1, -1]
])

B = sp.Matrix([
    4,
    1,
    0
])

```

3.6.4 Giải hệ bằng phương pháp Gauss–Jordan

Một cách trực tiếp để giải hệ là đưa ma trận mở rộng về dạng bậc thang rút gọn.
Ma trận mở rộng được tạo bởi:

```
Ab = A.row_join(B)
```

Sau đó sử dụng:

```
R, pivot_columns = Ab.rref()
```

Toàn bộ chương trình:

```

import sympy as sp

A = sp.Matrix([
    [1, 2, 1],
    [2, -1, 1],
    [1, 1, -1]
])

B = sp.Matrix([
    4,
    1,
    0
])

Ab = A.row_join(B)

R, pivot_columns = Ab.rref()

print("Ma tran mo rong:")
print(Ab)

print("Dang bac thang rut gon:")
print(R)

print("Cac cot tru:")
print(pivot_columns)

```

Kết quả dạng bậc thang rút gọn là

$$\begin{pmatrix} 1 & 0 & 0 & \frac{1}{3} \\ 0 & 1 & 0 & \frac{10}{9} \\ 0 & 0 & 1 & \frac{13}{9} \end{pmatrix}.$$

Do đó nghiệm của hệ là

$$x = \frac{1}{3}, \quad y = \frac{10}{9}, \quad z = \frac{13}{9}$$

3.6.5 Giải hệ bằng gauss_jordan_solve()

SymPy cung cấp phương thức `gauss_jordan_solve()` để giải hệ phương trình dưới dạng ma trận.

Ta sử dụng:

```
solution, parameters = A.gauss_jordan_solve(B)
print(solution)
```

Với hệ đang xét, kết quả là

$$\begin{pmatrix} \frac{1}{3} \\ \frac{10}{9} \\ \frac{13}{9} \end{pmatrix}.$$

Do đó:

$$x = \frac{1}{3}, \quad y = \frac{10}{9}, \quad z = \frac{13}{9}.$$

3.6.6 Giải hệ bằng linsolve()

Một phương pháp khác là sử dụng hàm `linsolve()`.

Với ma trận hệ số A và ma trận B , có thể viết:

```
solution = sp.linsolve((A, B))
print(solution)
```

Kết quả:

```
{(1/3, 10/9, 13/9)}
```

Điều này tương ứng với

$$(x, y, z) = \left(\frac{1}{3}, \frac{10}{9}, \frac{13}{9} \right).$$

3.6.7 Kiểm tra nghiệm

Sau khi tìm được nghiệm, có thể thay nghiệm vào hệ để kiểm tra.

Ta khai báo:

```
x, y, z = sp.symbols('x y z')

solution = {
    x: sp.Rational(1, 3),
    y: sp.Rational(10, 9),
    z: sp.Rational(13, 9)
}

eq1 = x + 2*y + z
eq2 = 2*x - y + z
eq3 = x + y - z

print(eq1.subs(solution))
print(eq2.subs(solution))
print(eq3.subs(solution))
```

Kết quả lần lượt là

$$4, \quad 1, \quad 0.$$

Đây chính là các vế phải của hệ nên nghiệm đã tìm được là chính xác.

3.6.8 Xét số nghiệm bằng hạng ma trận

Để áp dụng định lý Rouché–Capelli, ta tính hạng của A và $\overline{A}$.

```
Ab = A.row_join(B)

print("rank(A) =", A.rank())
print("rank(Ab) =", Ab.rank())
```

Trong ví dụ trên:

$$\text{rank}(A) = \text{rank}(\overline{A}) = 3.$$

Hệ có 3 ẩn nên

$$\text{rank}(A) = \text{rank}(\overline{A}) = 3 = n.$$

Do đó hệ có nghiệm duy nhất.

3.6.9 Ví dụ hệ vô nghiệm

Xét hệ:

$$\begin{cases} x + y = 1, \\ 2x + 2y = 3. \end{cases}$$

Ta có

$$A = \begin{pmatrix} 1 & 1 \\ 2 & 2 \end{pmatrix}, \quad B = \begin{pmatrix} 1 \\ 3 \end{pmatrix}.$$

Trong Python:

```
import sympy as sp

A = sp.Matrix([
    [1, 1],
    [2, 2]
])

B = sp.Matrix([
    1,
    3
])

Ab = A.row_join(B)

print("rank(A) =", A.rank())
print("rank(Ab) =", Ab.rank())
```

Ta thu được

$$\text{rank}(A) = 1, \quad \text{rank}(\overline{A}) = 2.$$

Do đó

$$\text{rank}(A) < \text{rank}(\overline{A}).$$

Theo định lý Rouché–Capelli, hệ vô nghiệm.

Ta cũng có thể sử dụng:

```
print(sp.linsolve((A, B)))
```

Kết quả biểu thị tập nghiệm rỗng.

3.6.10 Ví dụ hệ có vô số nghiệm

Xét hệ:

$$\begin{cases} x + y = 2, \\ 2x + 2y = 4. \end{cases}$$

Ta có:

$$A = \begin{pmatrix} 1 & 1 \\ 2 & 2 \end{pmatrix}, \quad B = \begin{pmatrix} 2 \\ 4 \end{pmatrix}.$$

Trong Python:

```
import sympy as sp

A = sp.Matrix([
    [1, 1],
    [2, 2]
])

B = sp.Matrix([
    2,
```

```

    4
])

Ab = A.row_join(B)

print("rank(A) =", A.rank())
print("rank(Ab) =", Ab.rank())

print(sp.linsolve((A, B)))

```

Ta có

$$\text{rank}(A) = \text{rank}(\overline{A}) = 1 < 2.$$

Vì số ẩn là 2, hệ có vô số nghiệm.

Kết quả nghiệm có thể viết dưới dạng

$$(x, y) = (2 - t, t), \quad t \in \mathbb{R}.$$

3.7 Ví dụ tổng hợp:

Xét hệ phương trình:

$$\begin{cases} x + 2y - z = 2, \\ 2x + 4y - 2z = 4, \\ x + y + z = 3. \end{cases}$$

3.7.1 Bước 1. Thành lập ma trận

Ma trận hệ số:

$$A = \begin{pmatrix} 1 & 2 & -1 \\ 2 & 4 & -2 \\ 1 & 1 & 1 \end{pmatrix}.$$

Ma trận hệ số tự do:

$$B = \begin{pmatrix} 2 \\ 4 \\ 3 \end{pmatrix}.$$

Ma trận mở rộng:

$$\overline{A} = \begin{pmatrix} 1 & 2 & -1 & 2 \\ 2 & 4 & -2 & 4 \\ 1 & 1 & 1 & 3 \end{pmatrix}.$$

3.7.2 Bước 2. Thực hiện bằng SymPy

```

import sympy as sp

A = sp.Matrix([
    [1, 2, -1],
    [2, 4, -2],

```

```

    [1, 1, 1]
])

B = sp.Matrix([
    2,
    4,
    3
])

Ab = A.row_join(B)

print("Ma tran mo rong:")
print(Ab)

print("Hang cua A:", A.rank())
print("Hang cua ma tran mo rong:", Ab.rank())

R, pivot_columns = Ab.rref()

print("Dang bac thang rut gon:")
print(R)

print("Tap nghiem:")
print(sp.linsolve((A, B)))

```

Ta thu được:

$$\text{rank}(A) = \text{rank}(\overline{A}) = 2 < 3.$$

Do đó hệ có vô số nghiệm.

Kết quả có thể biểu diễn dưới dạng:

$$(x, y, z) = (4 - 3t, 2t - 1, t), \quad t \in \mathbb{R}.$$

3.7.3 Bước 3. Kiểm tra nghiệm

Thay

$$x = 4 - 3t, \quad y = 2t - 1, \quad z = t$$

vào phương trình thứ nhất:

$$x + 2y - z = (4 - 3t) + 2(2t - 1) - t.$$

Suy ra

$$x + 2y - z = 4 - 3t + 4t - 2 - t = 2.$$

Phương trình thứ hai:

$$2x + 4y - 2z = 2(4 - 3t) + 4(2t - 1) - 2t = 4.$$

Phương trình thứ ba:

$$x + y + z = (4 - 3t) + (2t - 1) + t = 3.$$

Như vậy nghiệm thu được thỏa mãn cả ba phương trình.

3.8 So sánh một số phương pháp trong SymPy

Trong quá trình giải bài toán ma trận và hệ phương trình tuyến tính, có thể sử dụng nhiều phương thức khác nhau của SymPy.

Phương thức	Mục đích	Kết quả
<code>rank()</code>	Tính hạng ma trận	Một số nguyên
<code>echelon_form()</code>	Dạng bậc thang	Ma trận
<code>rref()</code>	Dạng bậc thang rút gọn	Ma trận + cột trụ
<code>linsolve()</code>	Giải hệ tuyến tính	Tập nghiệm
<code>gauss_jordan_solve()</code>	Giải hệ tuyến tính	Nghiệm

Việc lựa chọn phương thức phụ thuộc vào mục đích của bài toán. Nếu cần xác định hạng, có thể sử dụng `rank()`. Nếu muốn quan sát quá trình biến đổi ma trận, có thể sử dụng `echelon_form()` hoặc `rref()`. Đối với bài toán giải hệ phương trình tuyến tính, `linsolve()` là một lựa chọn thuận tiện để thu được tập nghiệm.

3.9 Một số lưu ý khi sử dụng SymPy

Khi sử dụng SymPy để giải các bài toán về ma trận, cần lưu ý một số vấn đề sau.

Thứ nhất, chỉ số của hàng và cột trong Python bắt đầu từ 0. Vì vậy, hàng thứ nhất được ký hiệu bằng chỉ số 0, hàng thứ hai bằng chỉ số 1.

Thứ hai, các phép biến đổi trên ma trận có thể làm thay đổi trực tiếp ma trận ban đầu. Do đó, nếu muốn giữ lại ma trận ban đầu để đối chiếu, nên tạo một bản sao trước khi thực hiện phép biến đổi.

Ví dụ:

```
B = A.copy()
```

Sau đó thực hiện các phép biến đổi trên B thay vì A .

Thứ ba, khi làm việc với các phân số, nên để SymPy xử lý dưới dạng số hữu tỉ thay vì chuyển sang số thực gần đúng. Chẳng hạn:

```
sp.Rational(1, 3)
```

được biểu diễn chính xác dưới dạng

$$\frac{1}{3}.$$

Điều này đặc biệt quan trọng trong các bài toán đại số tuyến tính vì kết quả chính xác thường được yêu cầu.

3.10 Bài tập thực hành

1. Cho ma trận

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 2 \end{pmatrix}.$$

Hãy sử dụng **SymPy** để:

- (a) Đưa A về dạng bậc thang.
- (b) Đưa A về dạng bậc thang rút gọn.
- (c) Tìm hạng của A .

2. Cho ma trận

$$B = \begin{pmatrix} 1 & 2 & 0 \\ 2 & 4 & 1 \\ 3 & 6 & 1 \end{pmatrix}.$$

Hãy sử dụng **SymPy** để xác định hạng của ma trận.

3. Giải hệ phương trình

$$\begin{cases} x + y + z = 6, \\ 2x - y + z = 3, \\ x + 2y - z = 2. \end{cases}$$

4. Xét hệ

$$\begin{cases} x + y = 2, \\ 2x + 2y = 4. \end{cases}$$

Sử dụng hạng của ma trận để xác định số nghiệm của hệ.

5. Xét hệ

$$\begin{cases} x + y = 1, \\ 2x + 2y = 3. \end{cases}$$

Sử dụng **SymPy** để chứng minh hệ vô nghiệm.